

GPI-8510 4계층 폴더 구조 설명서

app · bsp · driver · service — 각 폴더의 역할을 쉽게 이해하기

대상 프로젝트 : GPI-8510 (축산 사일로 사료 계량기) | MCU : STM32F103ZET6 | 개발환경 : Keil MDK + StdPeriph

이 장비를 “무게를 재서 서버에 보내는 자동 로봇 식당”이라고 생각하면 이해가 쉽습니다. 우리가 만든 4개 폴더는 이 식당의 4개 층이며, 아래층으로 갈수록 “기계(하드웨어)”에 가깝고, 위층으로 갈수록 “생각(판단)”에 가깝습니다.

1. 한눈에 보는 4개 층

app	🗨️ 사장님(셰프) - 무엇을 할지 결정		← 제일 위 (머리)
service	👨‍🍳 준비팀 - 재료 다듬고 통역해서 전달		
driver	👉 손 - 기계(부품)를 직접 만짐		
bsp	📖 주방 배치도 - 뭐가 어느 다리(핀)에 꽂혔는지		← 제일 아래 (바닥)

위에서 아래로 “일을 시키고”, 아래에서 위로 “실행 결과를 올려주는” 구조입니다.

2. 각 폴더의 성격

- 📖 bsp/ — “주방 배치도(주소록)”
- 정체 : 부품이 몇 번 다리(핀)에 연결됐는지 적어둔 지도. 요리는 안 하고 위치만 알려줍니다.
 - 비유 : 냉장고·가스레인지가 주방 어디에 있는지 그린 배치도.
 - 실제 파일 : board_gpi8510.h — 딱 1개
 - ★ 새 PCB 때 : 부품 위치가 바뀌면 이 파일 하나만 고치면 됩니다. “3개월 → 7일”의 핵심!

driver/ — “기계를 직접 만지는 손”

- **정체** : 실제 부품을 직접 **켜고·끄고·읽는** 코드. 전기 신호를 다룹니다. 생각은 안 하고 시키면 실행합니다.
- **비유** : 칼로 자르고, 가스불을 켜고, 저울 눈금을 읽는 손.
- **실제 파일 (10개)** : `adc.c` (무게 센서 읽기), `lcd.c` / `lcd70.c` (화면), `usart.c` (통신선), `i2c_ee.c` (설정 저장 메모리), `sflash.c` (폰트 저장), `buzzer.c` (삐 소리), `led.c`, `key.c` (버튼), `timer.c` (시계)
- ★ **새 PCB 때** : 부품 종류가 같으면 그대로 재사용.

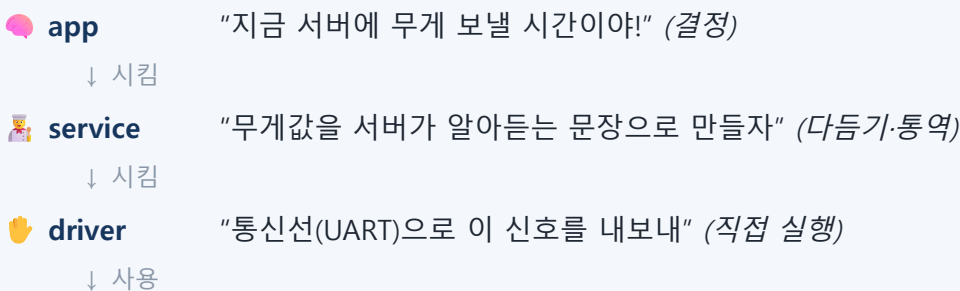
service/ — “준비팀(통역·중간다리)”

- **정체** : 손(driver)이 가져온 **날것 데이터를 다듬어** 위층(app)이 쓰기 좋게 만듭니다. 통신 규칙·대기줄(큐)·문장 해석 담당.
- **비유** : 야채를 손질해 셰프에게 넘기는 준비팀. 서버와 주고받는 **말(프로토콜)**을 통역하는 사람.
- **실제 파일 (10개)** : `tcpip_send.c` (서버로 보내기), `parsing.c` (받은 명령 해석), `queue.c` (순서 대기줄), `sms.c`·`zigbee.c`·`rs485.c` (각종 통신), `cdma_queue.c` (모뎀 송신줄), `agent_weight.c`·`agent_input_filter.c` (무게값 다듬기)
- ★ **새 PCB 때** : 통신 방식이 같으면 그대로 재사용. 재사용 핵심 자산.

app/ — “사장님(셰프)의 머리(판단)”

- **정체** : “무엇을 할지” 결정하는 두뇌. 언제 배출량을 계산하고, 언제 서버에 보내고, 화면에 뭘 띄울지.
- **비유** : 주문을 보고 “이 요리를 만들자!” 정하는 **셰프의 머리**. 직접 칼질은 안 하고 결정만 합니다.
- **실제 파일 (12개)** : `agent_discharge_calc.c` (배출량 계산), `agent_baichul_send.c` (배출정보 전송), `agent_cdma_task.c` (모뎀 처리), `agent_weight_display.c` (무게 화면), `agent_minute_event.c` (1분마다 할 일) 등 `Agent_*` 태스크
- ★ **새 PCB 때** : 새 보드에서 **달라진 요구사항만** 여기에 새로 작성. (예: 센서 1개 추가)

3. 실제로 어떻게 협력하나 — “무게 재서 서버 보내기”



4. 왜 4층으로 나뉘냐 (제일 중요)

한마디로 "안 바뀌는 것"과 "바뀌는 것"을 분리한 것입니다.

계층	새 PCB 나오면	왜?
 bsp	이것만 수정	부품 위치(핀)는 보드마다 바뀌니까
 app	달라진 것만 추가	요구사항은 조금씩 바뀌니까
 driver	그대로 재사용	부품 다루는 방법은 안 바뀌니까
 service	그대로 재사용	통신 규칙은 안 바뀌니까

예전엔 이 네 가지가 8,000줄 코드에 **다 뒤섞여** 있어서 새 보드마다 전부 뒤져야 했습니다(그래서 3개월).
이제 **바뀌는 부분(bsp + app)**만 손대면 되므로 **7일**로 줄어듭니다.